

Pattern Matching Algorithms

or

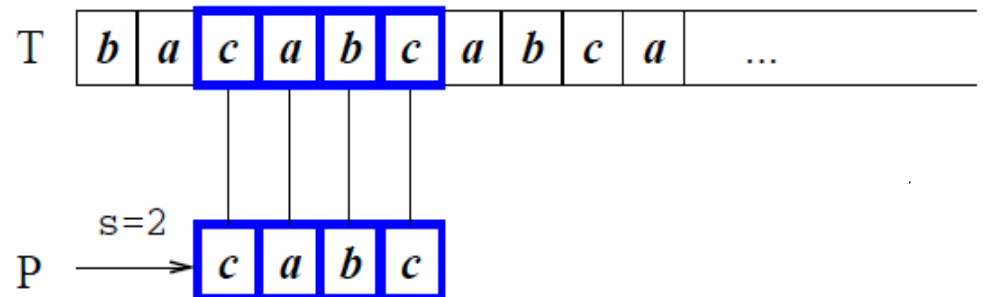
String Matching Algorithms

Pattern/ String Matching Problem and Terminology

Given a **text** array $T[1 \dots n]$ and a **pattern** array $P[1 \dots m]$ such that the elements of T and P are characters taken from alphabet Σ . e.g., $\Sigma = \{0, 1\}$ or $\Sigma = \{a, b, \dots, z\}$.

The **String Matching Problem** is to find *all* the occurrence of P in T .

A pattern P occurs with **shift** s in T , if $P[1 \dots m] = T[s + 1 \dots s + m]$. The String Matching Problem is to find all values of s . Obviously, we must have $0 \leq s \leq n - m$.



String Matching Applications

- Search Engine
- Plagiarism Detection
- Bioinformatics
- Spam Filters
- Spell Checkers
- Digital Forensics
- Information Retrieval Systems
- Intrusion Detection Systems

Brute-force search

- ▶ The aim: we would like to construct an algorithm that's capable of finding a pattern in a text
- ▶ Brute-force search is the naive approach ~ intuitive !!!
- ▶ Keep iterating through the text and if there is a mismatch we shift the pattern one step to the right
- ▶ Not so efficient especially when there are lots of matching prefixes
- ▶ For example: pattern is **DDDDDE** and the text is **DDDDDDDDDDDDDE**
- ▶ Main problem: needs backup for every mismatch... if there is a mismatch we jump back to the next character (!!!)
- ▶ Lots of compares: ~ $N * M$ where **N** is the length of text, **M** is the length of the pattern we are looking for
- ▶ Linear time guarantee would be better !!!

Brute-force search

Text T

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

Pattern P

T	E	S	T
---	---	---	---

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

It's a match

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

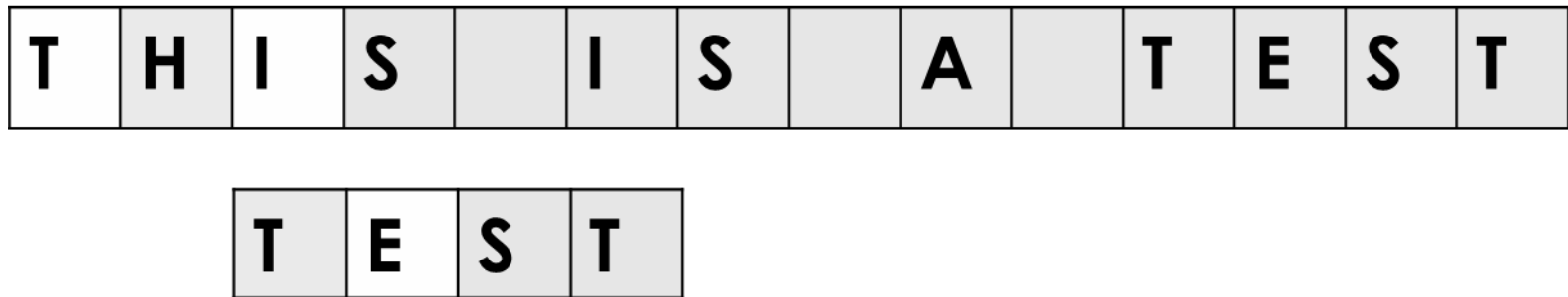
Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)



Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

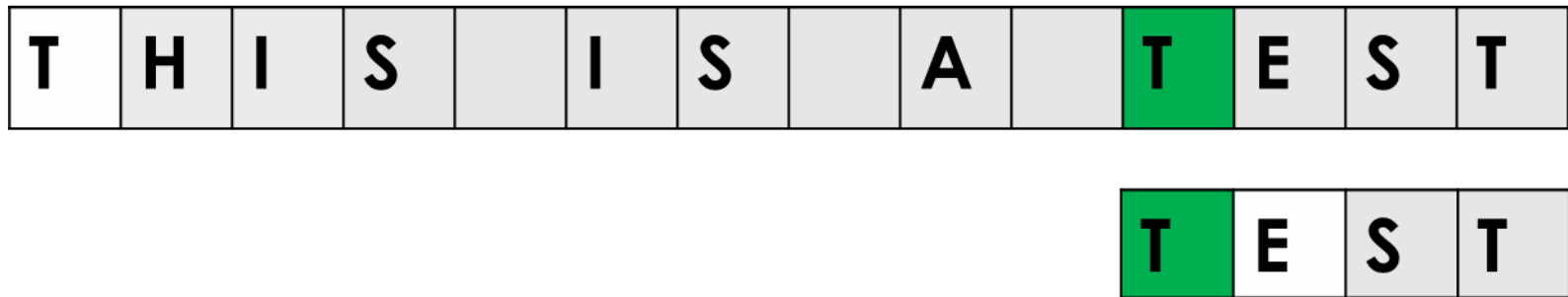
Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Mismatch occur

Brute-force Search (or Naïve Search)



Since there was a mismatch; therefore, shifting entire pattern window by 1 position in right direction to start comparison process again

It's a match

Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

It's a match

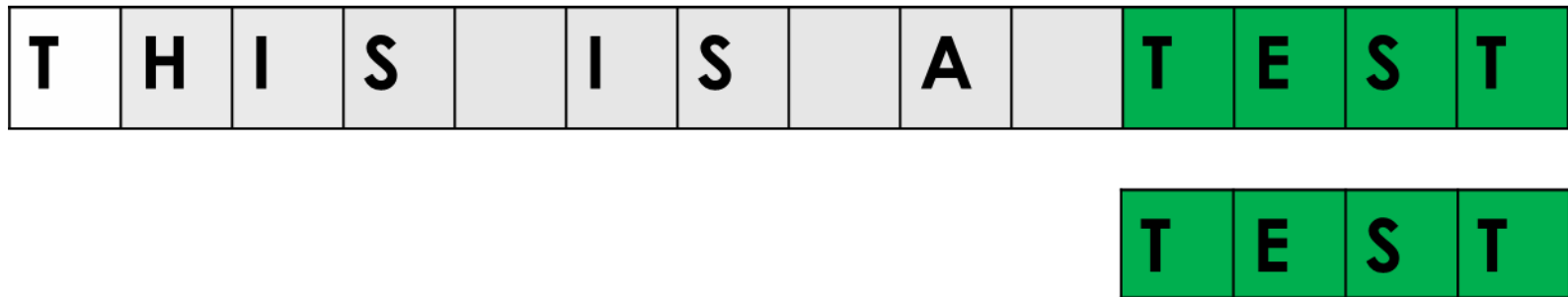
Brute-force Search (or Naïve Search)

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

It's a match

Brute-force Search (or Naïve Search)



It's a match finally

Output: The entire pattern P occurs with shift $s = 10$ in given Text T

Brute-Force or Naïve String Matching Algorithm (refer Corman Book)

The naive algorithm finds all valid shifts using a loop that checks the condition $P[1..m] = T[s + 1..s + m]$ for each of the $n - m + 1$ possible values of s .

NAIVE-STRING-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print "Pattern occurs with shift"  $s$ 
```

Space Complexity = $O(1)$ as no additional space required apart from input

Time Complexity = $O(n - m + 1) \times m = O(n \cdot m)$ **or** $\Theta(n^2)$ if $m = \lfloor n/2 \rfloor$

Naïve Algorithm Analysis

- It is very easy to implement but slow $O(nm)$ time complexity
- No preprocessing phase
- Space Complexity is $O(1)$ i.e. Constant
- Always shifts the window by exactly 1 position to the right; more no. of comparisons
- $O(n m)$ expected text characters comparisons in worst scenario

Problem with Naïve approach

Text T

D	D	D	D	D	D	S
---	---	---	---	---	---	---

Pattern P

D	D	D	S
---	---	---	---

Problem with Naïve approach



It's a match

Problem with Naïve approach



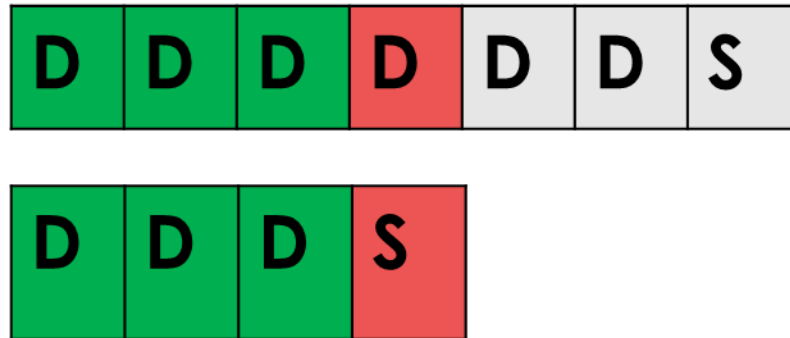
It's a match

Problem with Naïve approach



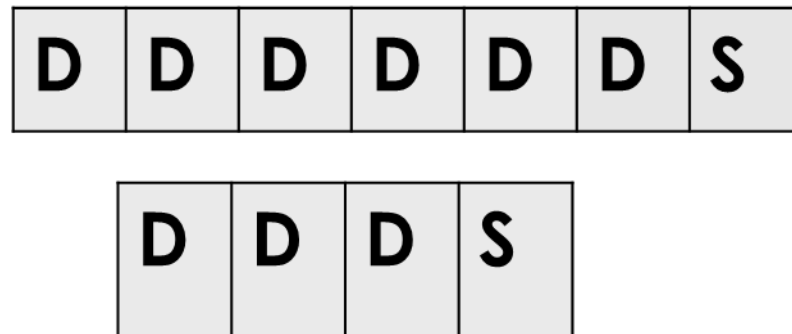
It's a match

Problem with Naïve approach



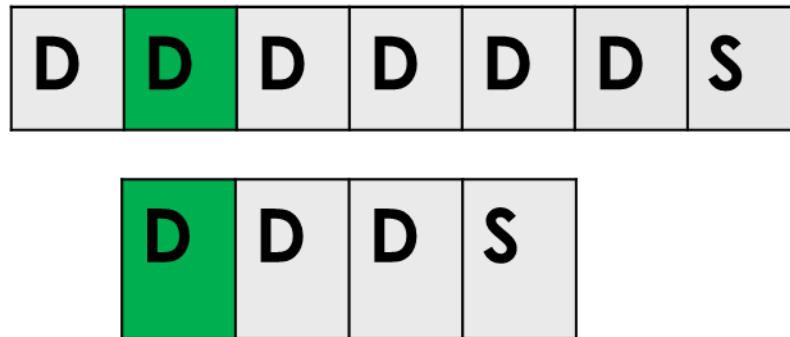
Mismatch occur for last character comparison

Problem with Naïve approach



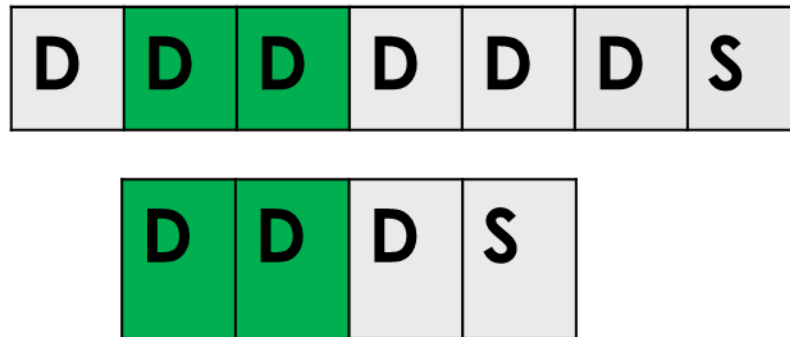
Shifting the entire pattern window by 1 position in right direction to start comparison process again

Problem with Naïve approach



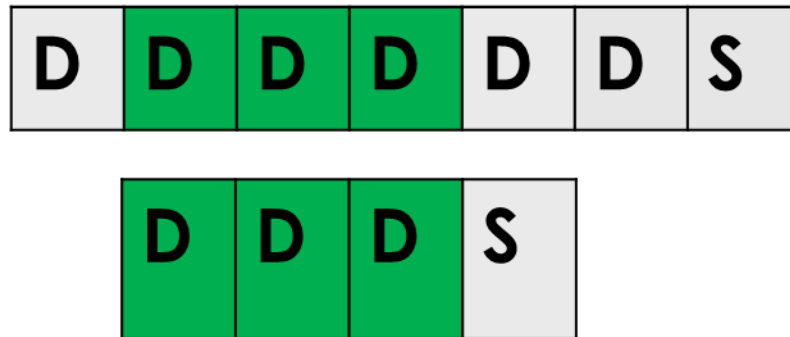
It's a match

Problem with Naïve approach



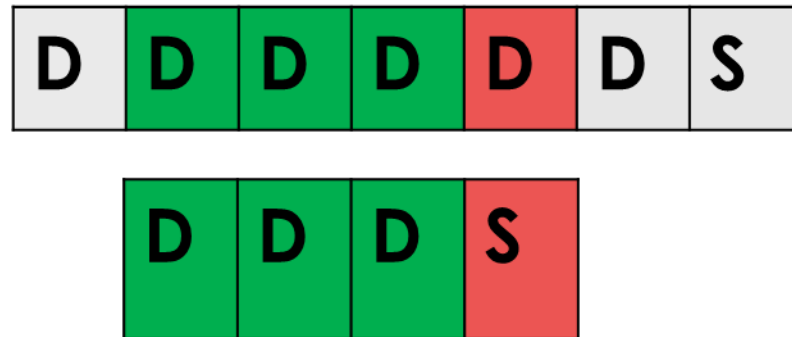
It's a match

Problem with Naïve approach



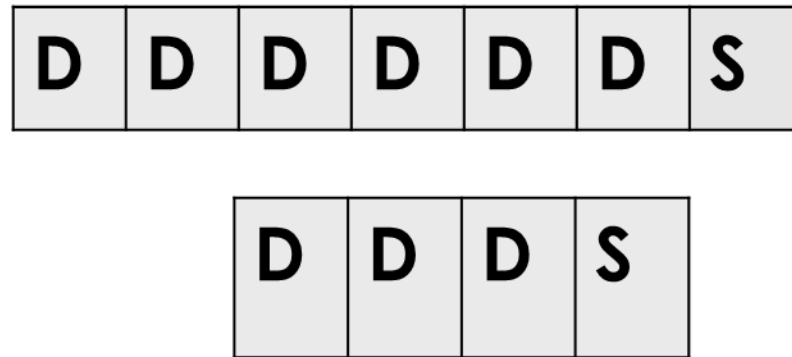
It's a match

Problem with Naïve approach



Mismatch occur for last character comparison again

Problem with Naïve approach



Since there was mismatch happened; hence we are shifting the entire pattern window again by 1 position in right direction to start comparison process again

Problem with Naïve approach



It's a match

Problem with Naïve approach



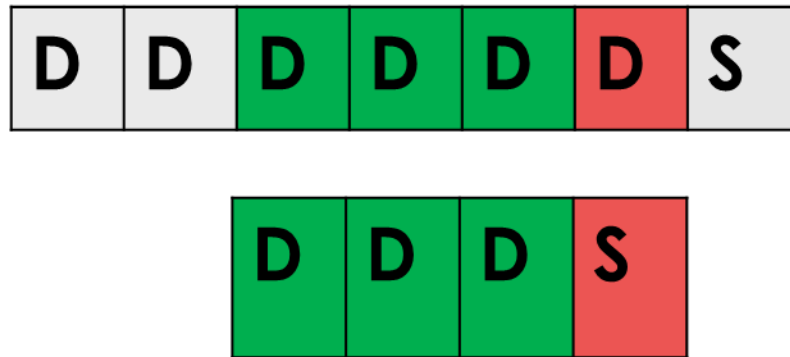
It's a match

Problem with Naïve approach



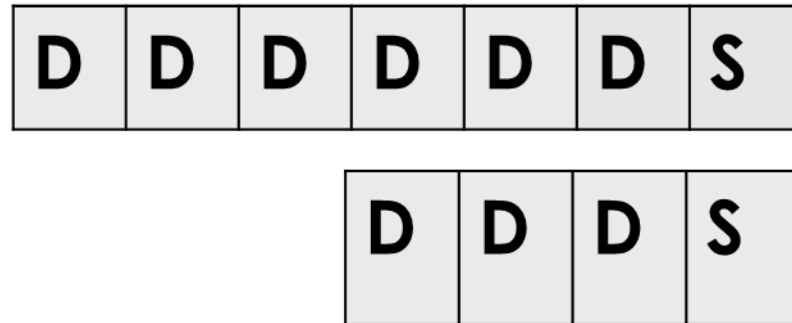
It's a match

Problem with Naïve approach



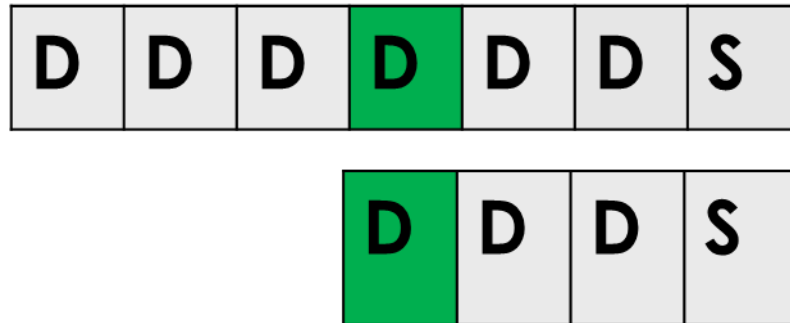
Mismatch occur for last character comparison again

Problem with Naïve approach



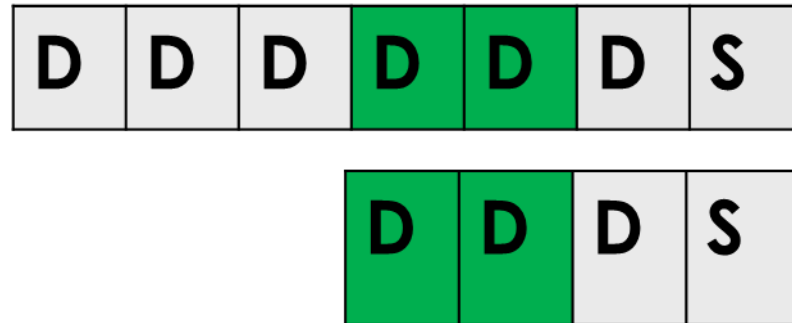
Shifting the entire pattern window by 1 position in right direction to start comparison process again

Problem with Naïve approach



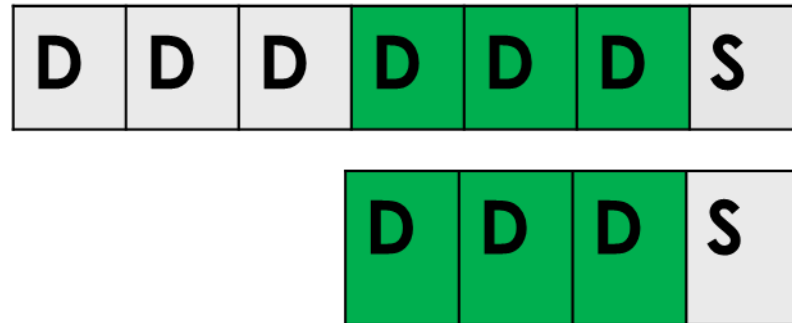
It's a match

Problem with Naïve approach



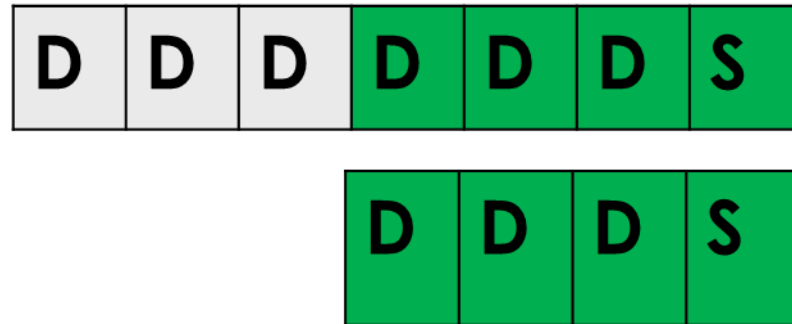
It's a match

Problem with Naïve approach



It's a match

Problem with Naïve approach



It's a match finally

Output: The entire pattern P occurs with shift $s = 3$ in given Text T

Naïve approach is not an efficient pattern matching algorithm due to its high no. of comparisons

Boyer-Moore Algorithm

Boyer-Moore search



- ▶ Problem with brute force search → we keep considering too many bad options as well ~ maybe we can eliminate a lot of possibilities
- ▶ That's why Boyer-Moore algorithm came to be
- ▶ Very efficient string search algorithm
- ▶ The algorithm needs to preprocess the pattern, but not the whole text !!!
- ▶ The algorithm runs faster as the length of the pattern increases
- ▶ The key features of the algorithm are to match on the tail of the pattern rather than the head
- ▶ Why is it good? We can skip multiple characters at the same time rather than searching every single character in the text

bad match table”

- ▶ Make a table of the characters
- ▶ Make sure the table does not contains repetitive characters // if there is several **a** letters in the pattern, the bad table only contains one **a** letter
- ▶ **Max(1, lengthOfPattern-actualIndex)** // this is the formula we use
- ▶ We iterate over the pattern and compute the values to the bad match table → we keep updating the old values for the same characters !!!

bad match table"

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

$\max(1, \text{lengthOfPattern} - \text{indexOfActualCharacter} - 1)$

Letters	T	E	S	*
Values				

bad match table”

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

$\max(1, \text{lengthOfPattern} - \text{indexOfActualCharacter})$

Letters	T	E	S	*
Values	3			

bad match table"

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

$\max(1, \text{lengthOfPattern} - \text{indexOfActualCharacter})$

Letters	T	E	S	*
Values	3	2		

bad match table”

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

$\max(1, \text{lengthOfPattern} - \text{indexOfActualCharacter} \cdot \quad)$

Letters	T	E	S	*
Values	3	2	1	

bad match table"

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

$\max(1, \text{lengthOfPattern} - \text{indexOfActualCharacter})$

Letters	T	E	S	*
Values	1	2	1	

bad match table"

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

$\max(1, \text{lengthOfPattern} - \text{indexOfActualCharacter})$

Letters	T	E	S	*
Values	1	2	1	4

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
T	E	S	T										

Letters	T	E	S	*
Values	1	2	1	4

Mismatch for char S in Text T, hence according to the table value **we have to shift entire pattern window by 1 position** in right direction to start comparison process again

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Mismatch for char * in Text T, hence according to the table value **we have to shift entire pattern window by 4 position** in right direction to start comparison process again

Boyer-Moore search

T H I S I S A T E S T

T E S T

Letters	T	E	S	*
Values	1	2	1	4

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Mismatch for char A in Text T (i.e. *), hence according to the table value **we have to shift entire pattern window by 4 position** in right direction to start comparison process again

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Mismatch for char S in Text T, hence according to the table value **we have to shift entire pattern window by 1 position** in right direction to start comparison process again

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

Boyer-Moore search

T	H	I	S		I	S		A		T	E	S	T
---	---	---	---	--	---	---	--	---	--	---	---	---	---

T	E	S	T
---	---	---	---

Letters	T	E	S	*
Values	1	2	1	4

It's a match

Boyer-Moore search

T H I S I S A T E S T

T E S T

Letters	T	E	S	*
Values	1	2	1	4

It's a match

Boyer-Moore search

T H I S I S A T E S T

T E S T

Letters	T	E	S	*
Values	1	2	1	4

It's a match

Boyer-Moore search



Letters	T	E	S	*
Values	1	2	1	4

It's a match again finally

add all those shifts we have performed, i.e. $1+4+4+1 = 10$ shifts

Output: The entire pattern P occurs with shift $s = 10$ in given Text T

Boyer-Moore's Algorithm

BoyerMoore(T, P)

1. $L \leftarrow$ last occurrence array computed from P
2. $i \leftarrow 0$ // current guess
3. **while** $i \leq n - m$
4. **for** ($j \leftarrow m - 1, j \geq 0, j--$)
5. **if** $T[i + j] \neq P[j]$ **break**
6. **if** $j = -1$ **return** "found at guess i "
7. **else** $i \leftarrow i + \max\{1, j - L[T[i + j]]\}$
8. **return** FAIL

COMPUTE-LAST-OCCURRENCE-FUNCTION (P, m, Σ)

1. **for** each character $a \in \Sigma$
2. do $L[a] = 0$
3. **for** $j \leftarrow 1$ to m
4. do $L[P[j]] \leftarrow j$
5. **Return** L

Analysis of Boyer-Moore's Algorithm

Analysis

- ▶ Turns out to be very efficient !!!
- ▶ Mismatched character heuristics takes about $\sim N / M$ character compares, where **M** is the length of the pattern and **N** is the length of the text
- ▶ It is not even linear: it is sublinear
- ▶ So the longer the pattern $\rightarrow$ the faster the algorithm become
- ▶ BUT worst case scenario can as bad as brute force when for example pattern is like **CDDDDDD** and text is **DDDDDDDDDDDD**

Various Pattern Matching Algorithm Complexity Analysis

m – the length (size) of the pattern; n – the length of the searched text

String search algorithm	Time complexity for	
	preprocessing	matching
Naïve	0 (none)	$\Theta(n \cdot m)$
Rabin-Karp	$\Theta(m)$	avg $\Theta(n + m)$ worst $\Theta(n \cdot m)$
Finite state automaton	$\Theta(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$
Boyer-Moore	$\Theta(m + \Sigma)$	$\Omega(n/m), O(n)$
Bit based (approximate)	$\Theta(m + \Sigma)$	$\Theta(n)$